RETAIL INVENTORY MANAGEMENT SYSTEM

A practical workspace for catalog, inventory, supplier and reporting operations

LEARNER NAME	Jasmeet Singh	**PROJECT CODE**	PRG_RIMS_001
CLIENT	XYZ Retail Corporation	**DOMAIN**	Retail Inventory Management
PROGRAMME	Individual Project	**FORMAT**	Full-stack Web Application
TECHNOLOGY	React, Spring Boot, PostgreSQL	**REPORT DATE**	30 October 2026

PROJECT SCOPE

Dashboard KPIs, product and supplier management, inventory thresholds, stock movement history, and downloadable inventory CSV summary.

Prepared for academic review

Executive Summary

Retail Inventory Management System (RIMS) is a browser-based inventory workspace for XYZ Retail Corporation. It brings core product catalog and stock operations into a single interface. The React frontend consumes Spring Boot REST endpoints; Spring Data JPA persists records in PostgreSQL for deployment, with file-backed H2 available for local development.

The implementation includes an operational dashboard, product and supplier screens, inventory review, stock transaction recording, and inventory CSV export. Low stock is identified when quantity is less than or equal to the configured minimum. Sample records are seeded on first database startup. This report describes the implemented system and available evidence; it does not claim PostgreSQL runtime verification or unrun tests.

> **IMPLEMENTATION TRANSPARENCY**
> Local verification used H2 and seeded examples. PostgreSQL settings and Docker Compose are present, but a live PostgreSQL run was not performed. If the API is unavailable, the UI displays demo sample data; fallback data is not persisted application data.

Report contents

SECTION	REPORT CONTENT
01	Introduction and Problem Statement
02	Proposed Solution and Requirements
03	System Architecture
04	Technology Stack and Project Structure
05	Database Design and Implementation
06	Functional Modules
07	Application Output Screenshots
08	Backend API Overview
09	Testing and Actual Results
10	Installation and Execution
11	Limitations and Future Enhancements
12	Conclusion and References

Project metadata

FIELD	DETAILS
Project title	Retail Inventory Management System (RIMS)
Project code / client	PRG_RIMS_001 / XYZ Retail Corporation
Domain	Retail Inventory Management
Prepared by	Jasmeet Singh
Core stack	React + Vite / Java Spring Boot / PostgreSQL
Local development database	File-backed H2

1. Introduction and Problem Statement

1.1 Introduction

Retail operations balance product availability, replenishment, and accurate records. When catalog information and stock movements sit in disconnected spreadsheets, teams can lose visibility into current quantities and the history behind a change. RIMS provides an integrated interface for viewing and maintaining those records through a backend API.

1.2 Problem statement (5W1H)

PROMPT	PROJECT-SPECIFIC RESPONSE
Who?	Retail operations staff and inventory coordinators who maintain product and supplier records.
What?	Stock information can be fragmented, manual to update, and difficult to review against minimum levels.
When / where?	During catalog maintenance, stock receipt or deduction, and low-stock review.
Why?	A common inventory view improves visibility and preserves dated stock movement history.
How?	A responsive web interface backed by REST APIs and relational persistence.

1.3 Project objectives

- Present product count, units, estimated inventory value, and low-stock count on a dashboard.
- Provide product and supplier create, read, update, and delete operations.
- Record stock-in and stock-out events with transaction dates.
- Prevent a stock-out request that exceeds available quantity.
- Export an inventory snapshot as CSV and document setup.

1.4 Scope and intended use

RIMS is a small operational prototype for demonstration and local evaluation. Authentication, user roles, multi-location stock, and production backup policy are outside the current implementation.

2. Proposed Solution and Requirements

The solution uses a single-page React interface with module navigation. Axios sends requests to the Spring Boot API. Product, supplier, transaction, dashboard, and CSV operations are exposed under `/api`. The backend persists records through Spring Data JPA. For local startup without PostgreSQL, the default configuration uses H2 in file mode.

2.1 Functional requirements

MODULE	IMPLEMENTED CAPABILITY
Dashboard	Product count, total quantity, inventory value, low-stock count, category bars, recent transactions.
Product management	List, search by name/SKU, add, edit, and delete. Fields include SKU, name, category, price, description, quantity, minimum stock.
Inventory	Review on-hand quantity and minimum level; record stock-in and stock-out movements.
Supplier management	List, add, edit, and delete supplier contact records.
Stock transactions	Persist product snapshot, movement type, quantity, and timestamp; reject insufficient stock-out quantity.
Reports	Inventory summary screen and CSV download.

2.2 Non-functional requirements

The UI is responsive, uses a consistent light theme, and separates API calls from presentation. SKU uniqueness is enforced by the database. Inventory quantity is changed with its stock movement in a transactional method. Current implementation has minimal centralized validation and no login or authorization.

> **LOW-STOCK RULE**
> A product is low stock when quantity ≤ minStock. This check drives dashboard and inventory status.

3. System Architecture

A user interacts with the React application in a browser. The frontend sends JSON requests to Spring Boot controllers. Controllers use Spring Data repositories to load and save JPA entities. PostgreSQL is configured for container deployment and H2 for local development.

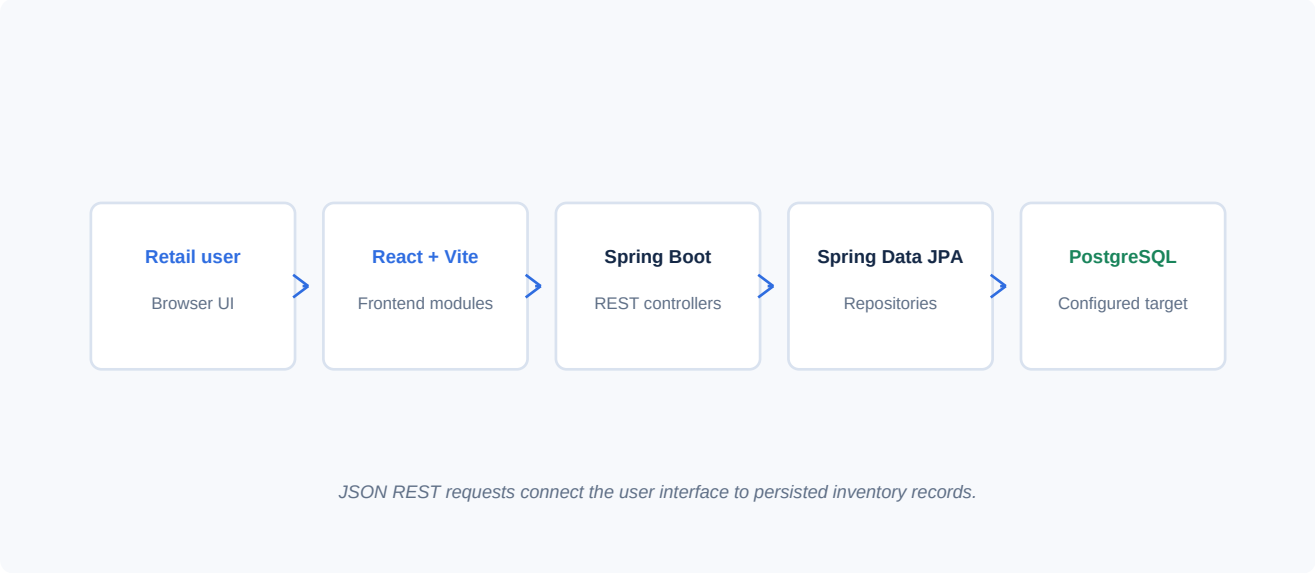


Figure 1. RIMS logical architecture and application data flow.

3.1 Request flow

1. The user opens a module or submits a form.
2. Axios sends an HTTP request to `/api`.
3. A controller reads or updates JPA-backed records.
4. Database changes persist; stock movement updates quantity and history in a transactional method.
5. The response refreshes a table, dashboard, or report view.

3.2 Architecture note

Controllers currently access repositories directly. The original target architecture called for a separate service and DTO layer; those layers are not present in this implementation. Introducing them with request validation is a clear next step.

4. Technology Stack and Project Structure

LAYER	TECHNOLOGY	ROLE
Frontend	React, Vite, JavaScript	Responsive module interface and production bundling.
HTTP / icons	Axios, Lucide React	REST communication and interface icons.
Backend	Java 17, Spring Boot 3.3, Spring Web	REST endpoints and application startup.
Persistence	Spring Data JPA / Hibernate	Entity persistence and schema generation.
Database	PostgreSQL 16; H2 local fallback	Relational target and local development store.
Packaging	Maven, Docker Compose	Java build and database/backend containers.
Reporting	ReportLab, Python	Generation of this academic report.

4.1 Workspace organization

```
.
+-- backend/              Spring Boot Maven service
|   +-- src/main/java/   Entities, repositories, controller, seed data
|   +-- src/main/resources/application.properties
+-- frontend/             React + Vite application
|   +-- src/main.jsx      Modules and UI interactions
|   +-- src/styles.css    Responsive styling
+-- docs/                  Architecture diagrams
+-- screenshots/           Captured application screens
+-- report/                PDF generation source
+-- output/pdf/             Final project report
+-- .env.example            Configuration template
+-- docker-compose.yml      PostgreSQL and backend services
+-- README.md               Setup and API documentation
```

4.2 Configuration overview

`VITE_API_URL` sets the frontend API base URL. `SPRING_DATASOURCE_URL`, `SPRING_DATASOURCE_USERNAME`, and `SPRING_DATASOURCE_PASSWORD` configure the backend datasource. Compose points the backend container to the PostgreSQL service.

5. Database Design and Implementation

5.1 Entity relationship overview

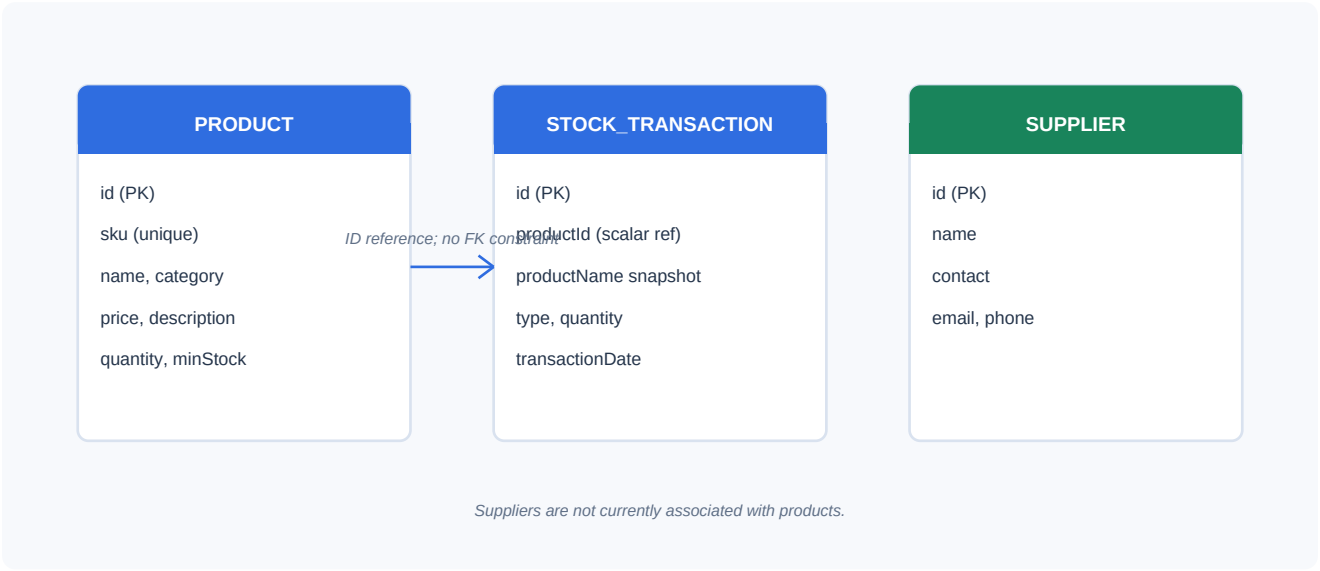


Figure 2. Implemented persistence fields and the scalar product reference recorded on stock transactions.

5.2 Entity summary

ENTITY	KEY FIELDS	NOTES
Product	id, sku, name, category, price, description, quantity, minStock	SKU is unique; quantity and threshold are stored on the product.
StockTransaction	id, productId, productName, type, quantity, transactionDate	Product ID is a scalar reference, not an enforced foreign key. Product name is retained as a snapshot.
Supplier	id, name, contact, email, phone	Supplier directory is independent of products.

5.3 Database behavior

Hibernate updates the schema from the entity model. On a clean database, seed data inserts four example products and two supplier records. File-backed H2 preserves local data across normal refreshes and restarts. PostgreSQL is configured in Docker Compose but was not started during this verification.

> **RELATIONSHIP LIMITATION**
> The diagram shows the product ID reference held by transaction history. There is no JPA relationship or database foreign key at present. Supplier records are not linked to products.

6. Functional Modules

6.1 Dashboard

Shows product count, total on-hand units, estimated retail inventory value, low-stock count, category quantity bars, and recent transaction rows. Values come from the backend while the API is available.

6.2 Product and inventory management

The catalog searches by product name, SKU, and category in the interface; the backend supports name/SKU search. Inventory view compares quantity to minimum stock and offers stock movement entry.

6.3 Supplier management

Supplier cards show name, contact person, email, and phone. The backend exposes supplier CRUD. Supplier-to-product assignment is not implemented.

6.4 Stock transaction history

A transaction request identifies product, movement (`IN` or `OUT`), and quantity. A stock-out that exceeds availability is rejected. A successful movement updates product quantity and creates a timestamped history record.

6.5 Reports

Reports displays an inventory summary and a CSV export action. The backend route returns SKU, product name, category, price, quantity, minimum stock, and stock status.

7. Application Output 1 - Dashboard

The dashboard summarizes the seeded local inventory. This capture shows four products, 132 units, an estimated value of $6,657, and one item at or below its minimum stock threshold.

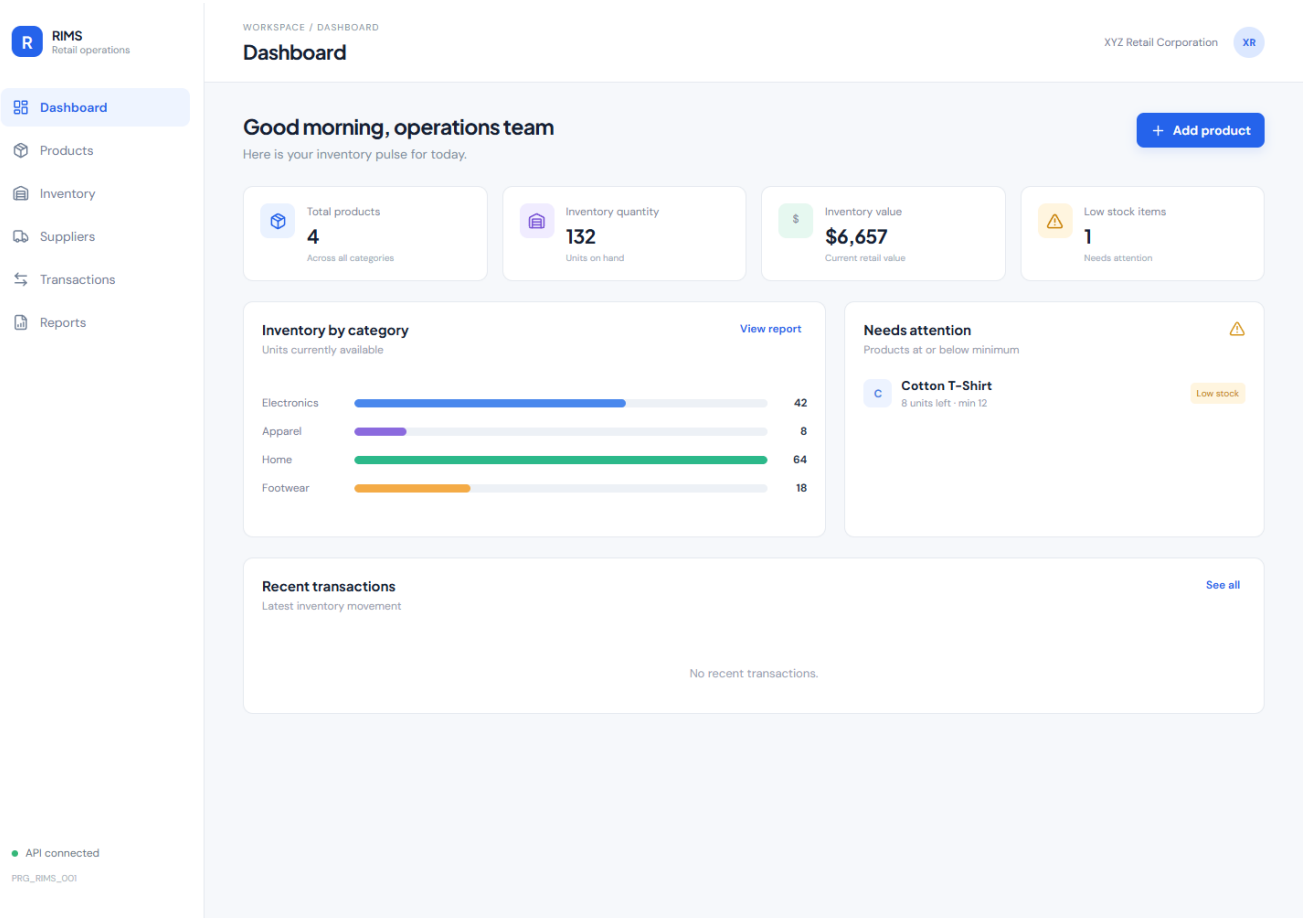


Figure 3. Running RIMS dashboard with the local Spring Boot API connected.

> **DASHBOARD VALUES**
> Values are from local seeded H2 records at capture time; they are examples, not live XYZ Retail Corporation inventory.

Application Output 2 - Product Catalog

The product catalog lists product identity, category, price, quantity on hand, and stock status. Search and row actions support routine catalog work.

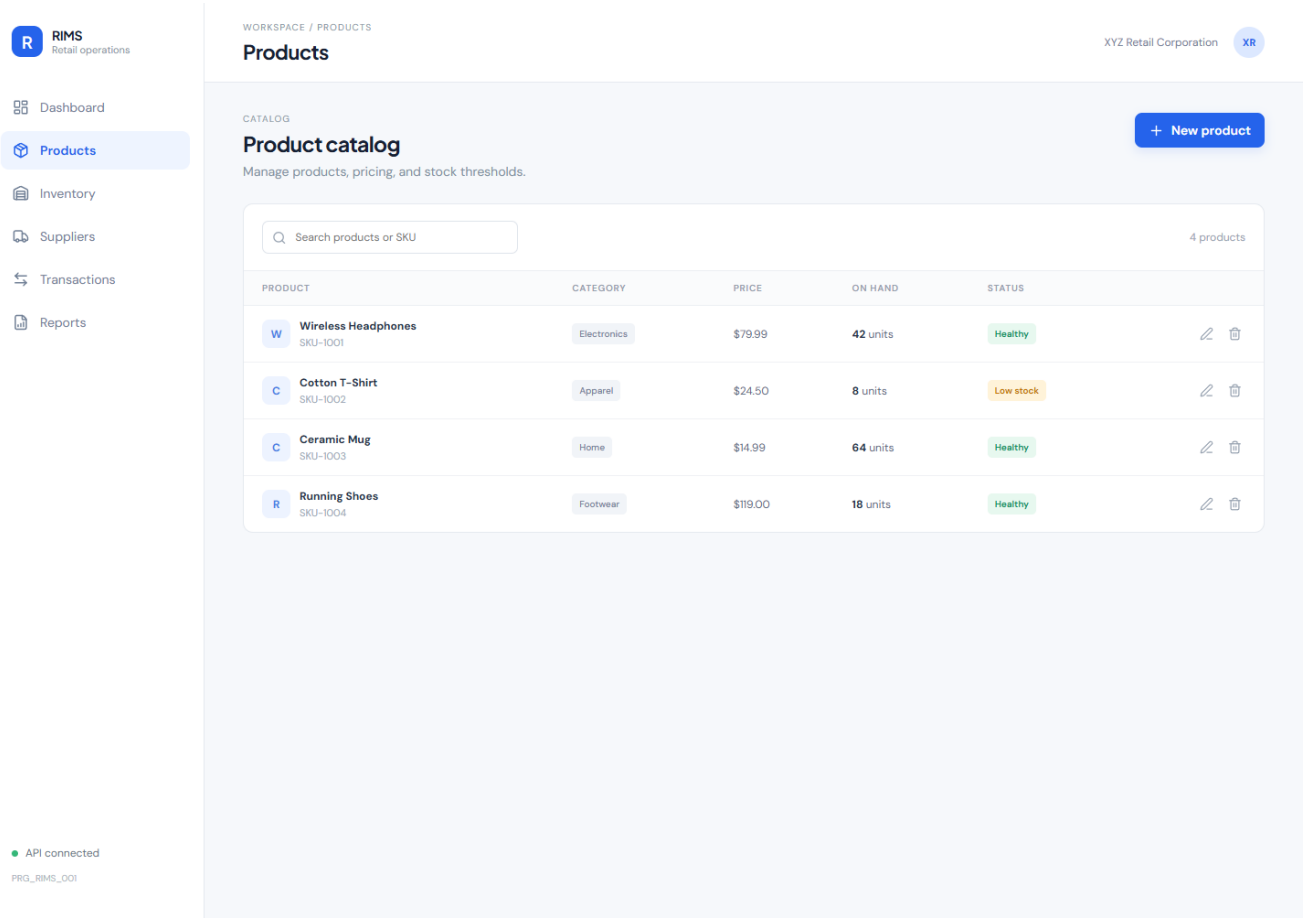


Figure 4. Product catalog with sample records and stock status labels.

PRODUCT DATA

The sample catalog is inserted by backend seed data on first startup. User changes are submitted through the backend API.

Application Output 3 - Inventory Levels

The inventory view provides a focused review of available units, configured minimums, and stock coverage.

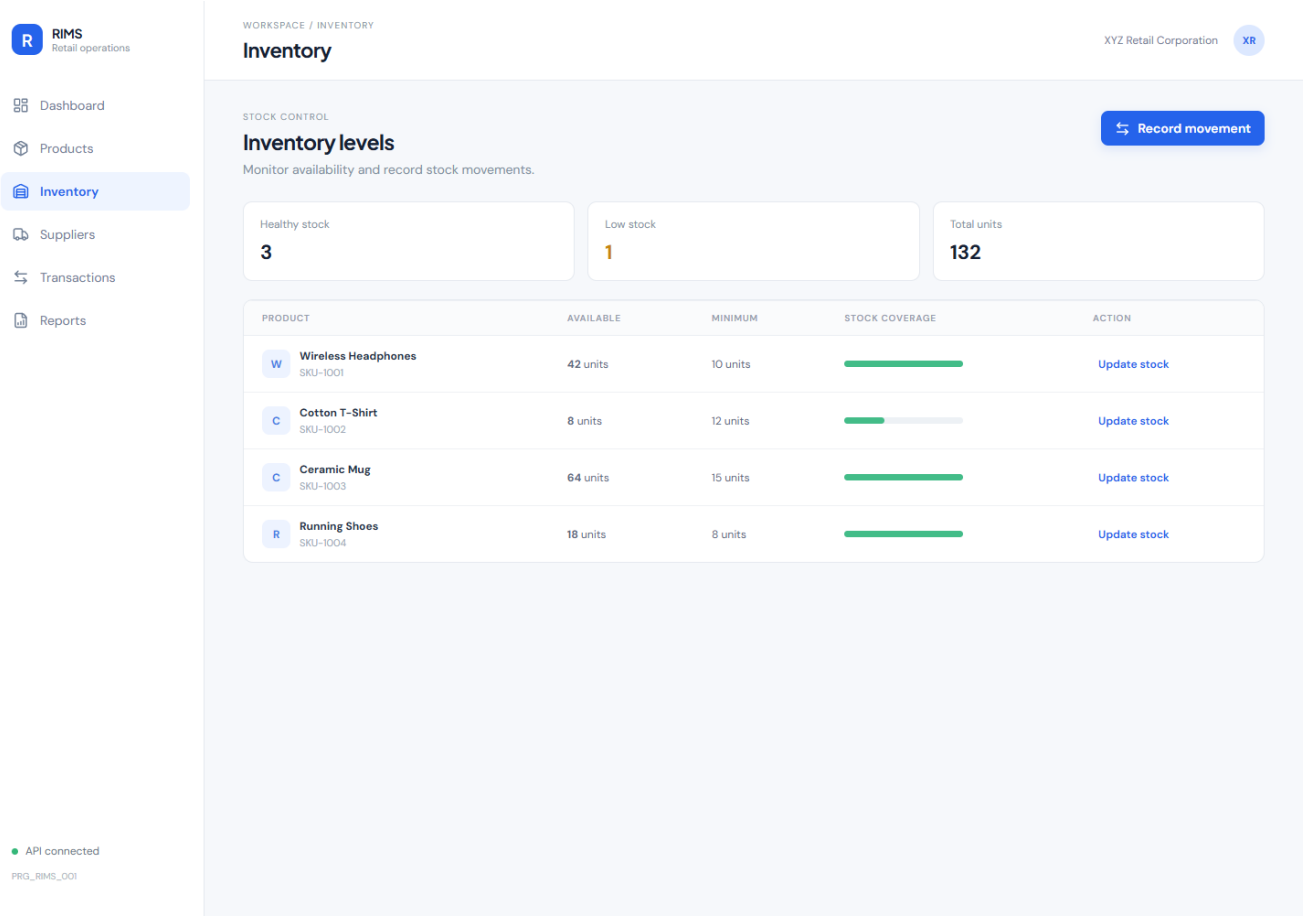


Figure 5. Inventory levels with quantities and minimum thresholds.

STOCK CONTROL

Low-stock status uses the stored quantity and minimum stock fields. Transaction dates are assigned by the backend when recorded.

Application Output 4 - Supplier Directory

Supplier cards group supplier names and contact channels. The screen offers create, edit, and delete controls.

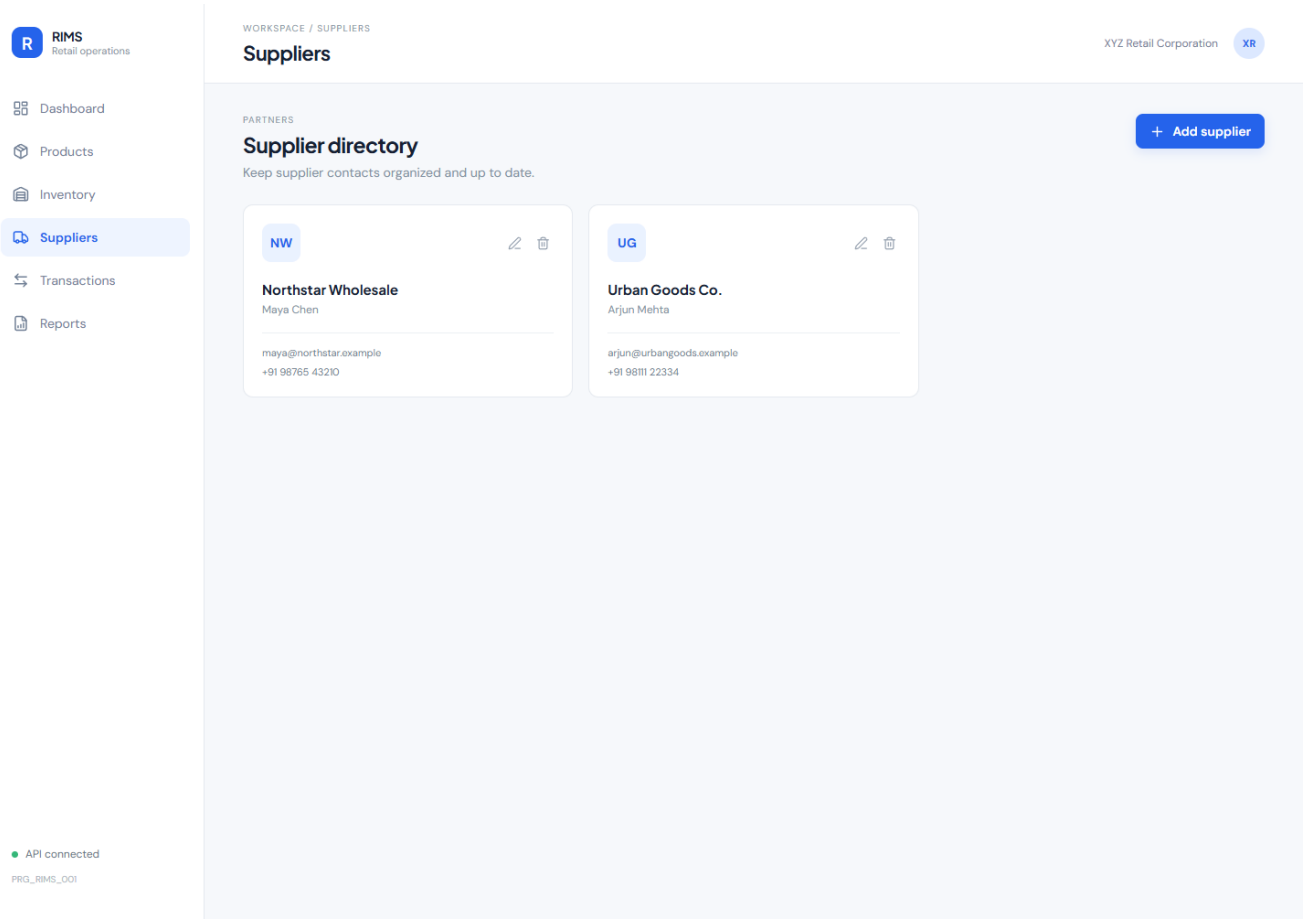


Figure 6. Supplier directory from the running RIMS frontend.

SUPPLIER SCOPE

Supplier records currently remain an independent directory and are not linked to individual products.

Application Output 5 - Reports and Export

The reports screen presents an inventory summary and CSV export action.

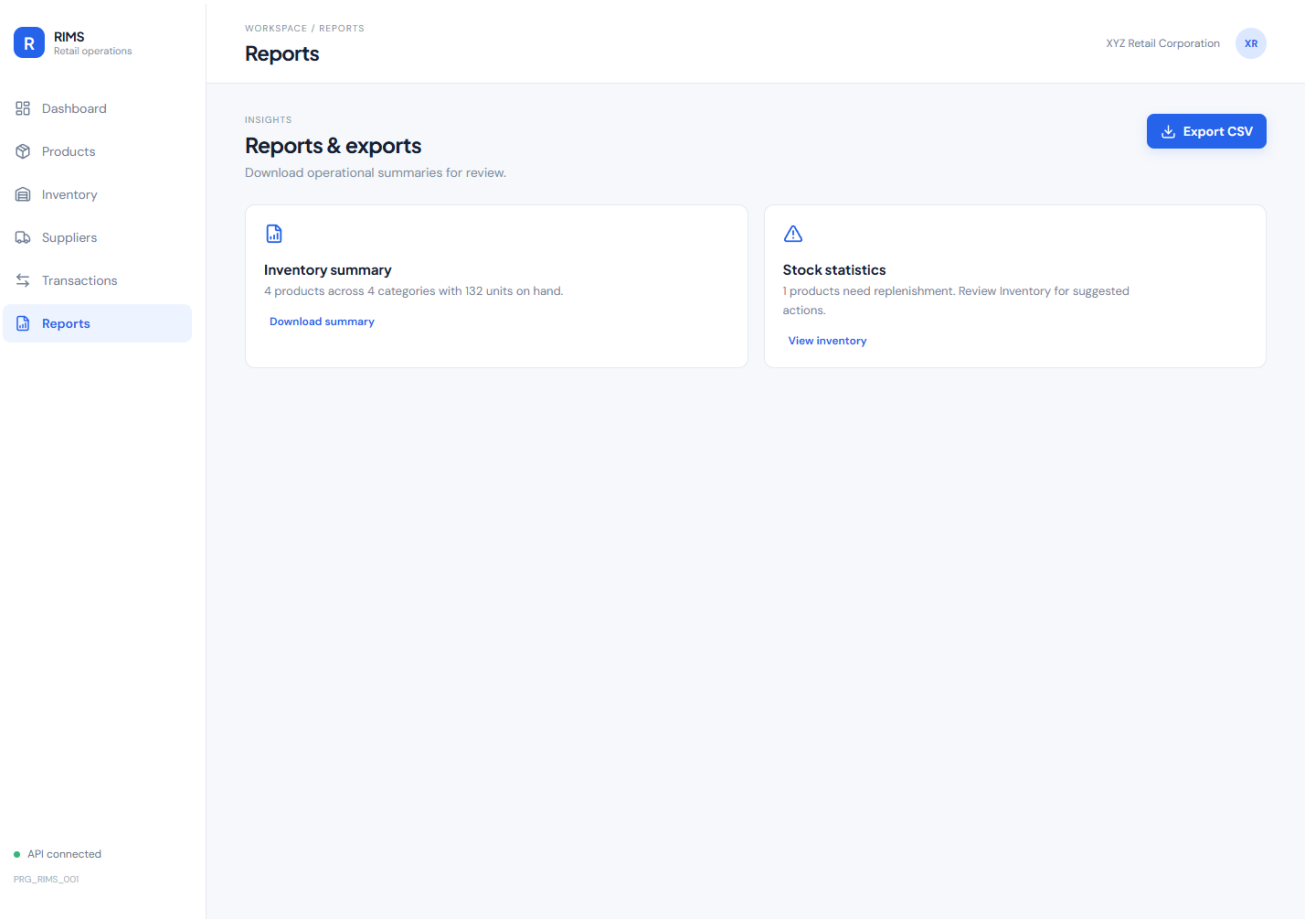


Figure 7. Reports screen with inventory CSV export.

EXPORT NOTE

CSV is a point-in-time inventory listing, not a movement trend or supplier performance report.

8. Backend API Overview

The backend exposes REST endpoints under `/api`. The frontend default base URL is `http://localhost:8080/api`.

METHOD	ENDPOINT	PURPOSE
GET	/api/products	List products; optional search query.
POST	/api/products	Create product.
PUT	/api/products/{id}	Update product.
DELETE	/api/products/{id}	Delete product.
GET / POST	/api/suppliers	List or create supplier records.
PUT / DELETE	/api/suppliers/{id}	Update or delete a supplier.
GET	/api/transactions	List stock movement history.
POST	/api/transactions	Record stock IN/OUT and update quantity.
GET	/api/dashboard	Return KPIs and recent transactions.
GET	/api/reports/inventory.csv	Download inventory CSV.

8.1 Example transaction payload

```
{
  "productId": 2,
  "type": "OUT",
  "quantity": 3
}
```

A successful response contains the transaction ID, product ID and name snapshot, type, quantity, and timestamp. Stock-out requests above available quantity are rejected. The API does not yet provide a centralized error response format.

9. Testing and Actual Results

The following checks were run while preparing the project and report. Results describe this local environment and are not a production deployment.

CHECK	OBSERVED RESULT
Frontend production build	Passed: Vite production bundle completed.
Backend Maven package	Passed: `mvn package -DskipTests`. No backend unit test suite is present.
Backend startup	Passed locally with H2; JPA initialized and seed records were served.
Dashboard API	Passed: 4 products, 132 units, 1 low-stock item returned.
Product API CRUD	Passed smoke check: create, update, and delete a temporary test product.
Supplier API	Create and delete smoke checks passed; update path was not separately exercised.
CSV endpoint	Passed: returned header and product rows including low-stock status.
Stock movement persistence	Implemented; dedicated live movement and restart-persistence test was not run for this report.
PostgreSQL runtime	Not verified: a PostgreSQL container/service was not started.
PDF integrity	Verified: generated PDF reopened, text extracted, and five application screenshots embedded.

TEST DATA CLEANUP

Temporary product and supplier records used for API smoke checks were deleted. Seed product and supplier records remained unchanged.

9.1 Verification limits

No automated frontend or backend tests were executed. The Maven package command skipped tests. Screenshots were captured from the running frontend using local seeded data.

10. Installation and Execution

10.1 Prerequisites

Node.js 18+ and npm; Java 17+; Maven 3.9+; PostgreSQL 14+ or Docker Desktop for the PostgreSQL deployment path.

10.2 Start backend with local H2

```
cd backend
mvn spring-boot:run
```

The default file-backed H2 database is configured in `backend/src/main/resources/application.properties`. The backend listens on port 8080.

10.3 Start frontend

```
cd frontend
npm install
npm run dev
```

Open `http://localhost:5173`. Configure `VITE_API_URL` if the API is hosted elsewhere.

10.4 Start PostgreSQL and backend with Docker

```
docker compose up --build
```

Compose starts PostgreSQL and builds the backend container. The frontend remains a separate Vite process in the documented local workflow. For local backend runs, set the `SPRING_DATASOURCE_*` values from `.env.example`.

> **ENVIRONMENT CONFIGURATION**
> The checked-in `.env.example` contains local demonstration credentials. Replace them with deployment-specific values before connecting to a shared database.

11. Limitations and Future Enhancements

11.1 Current limitations

• Authentication and user roles are not implemented.
• Forms have minimal validation; backend request validation is not applied consistently.
• Controllers access repositories directly; service and DTO layers are absent.
• Transactions store product ID as a scalar, not an enforced foreign key.
• Suppliers are not linked to products, purchase orders, or replenishment.
• PostgreSQL configuration exists but live connectivity was not verified.
• When the API is offline, the UI shows fallback sample data; offline edits are not persisted to the backend.
• Reporting is limited to an inventory CSV; there are no movement trends or supplier analytics.

11.2 Future enhancements

• Add Spring Security authentication and role-based access.
• Separate controller, service, repository, entity, and DTO responsibilities; add consistent validation and error responses.
• Add enforced product-transaction relationships, audit user identity, and transaction pagination.
• Associate suppliers with products and add purchase orders and replenishment workflows.
• Add automated API/UI tests and verify PostgreSQL persistence through Compose.
• Expand reports with date filters, movement summaries, and warehouse/location support.

12. Conclusion

RIMS provides a working foundation for retail inventory operations: product and supplier management, quantity thresholds, stock movement recording, dashboard summaries, and CSV export. The application was built and exercised locally with H2 sample data. Its documented PostgreSQL setup and clearly stated limitations provide a basis for academic review and continued development.

References

[1] Spring Boot Reference Documentation: https://docs.spring.io/spring-boot/
[2] Spring Data JPA Reference Documentation: https://docs.spring.io/spring-data/jpa/reference/
[3] PostgreSQL Documentation: https://www.postgresql.org/docs/
[4] React Documentation: https://react.dev/
[5] Vite Documentation: https://vite.dev/
[6] User-supplied Student Analyzer report; referenced for presentation structure and visual style, not project-specific facts.